

TRABAJO FINAL INTEGRADOR

Aplicación Java con relación 1→1 unidireccional + DAO + MySQL

Alumnos:

- *Montero Morinigo, Ramón Armando*
- *Capelli, Camilo*

ENLACE REPOSITORIO: [GitHub - RamaMontero/Programaci-n-II---UTN-TUPaD: Presentación de trabajos prácticos de la materia Programación II](#)

(CARPETA: Prog2_TFI - MONTERO Y CAPELLI)

Link video: <https://youtu.be/WYDWzjLUly4>

Materia: Programación II

Profesor/a: Cinthia Rigoni

Tutor: Giuliano Espejo

COMISIÓN 11

INDICE:

1. Diseño del Modelo de Dominio y Decisiones de Modelado
2. Diseño UML y decisiones
 - 2.1. Decisiones de Modelado
 - 2.2. Justificación de la Relación 1→1 mediante FK única
3. Modelado Java de Entidades
 - 3.1. Estructura de Paquetes
 - 3.2. Decisiones de Modelado
 - 3.3. Código de las clases
4. Persistencia y conexión a la base de datos (config/)
 - 4.1. Archivo de configuración: db.properties
 - 4.2. Clase de conexión: ConexionBD.java
 - 4.3. Prueba de funcionamiento
 - 4.4. Resultado obtenido
5. DAO (Data Access)
 - 5.1 Descripción de los DAOs
 - 5.2. Métodos CRUD implementados
 - 5.3. Código de los DAOs
 - 5.4. Conclusión
6. Service (Transacciones)
 - 6.1 Descripción del Servicio
 - 6.2. Simulación de Rollback por CUIT Duplicado
 - 6.3. Conclusión
7. AppMenu (Interfaz de Consola)
 - 7.1. Funcionalidades del AppMenu
 - 7.2. Conclusión y posibles mejoras a futuro

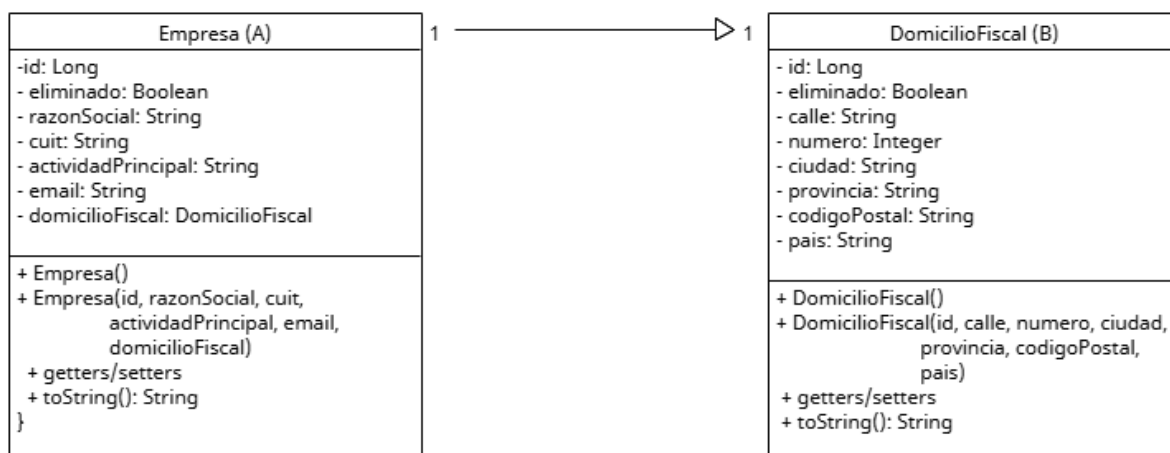
ANEXO:

Roles de los integrantes, fuentes utilizadas y conclusión final.

1. Diseño del Modelo de Dominio y Decisiones de Modelado

Para este Trabajo Final Integrador se diseñó un modelo de dominio basado en una relación unidireccional 1→1 entre las entidades Empresa (A) y DomicilioFiscal (B). La clase Empresa contiene una referencia a la clase DomicilioFiscal, mientras que DomicilioFiscal no conoce a Empresa, cumpliendo así con el requisito de direccionalidad definida en la consigna.

2. Diseño UML y decisiones



La relación modelada es 1 a 1, donde cada empresa posee exactamente un domicilio fiscal, y cada domicilio fiscal pertenece a una única empresa. En el UML esto se refleja con multiplicidad 1..1 en ambos extremos y flecha desde Empresa hacia DomicilioFiscal para denotar la unidireccionalidad.

2.1. Decisiones de Modelado

La relación entre las clases se modela como unidireccional desde Empresa hacia DomicilioFiscal, ya que la consigna indica explícitamente que la dirección debe ser $A \rightarrow B$. Esto significa que la clase Empresa incluye un atributo del tipo DomicilioFiscal, mientras que DomicilioFiscal no mantiene ninguna referencia de vuelta hacia Empresa. De esta forma se evita un acoplamiento innecesario, se elimina la doble referencia y se mantiene una interpretación más clara y limpia del modelo.

Ambas entidades incluyen el atributo eliminado, que funciona como un indicador de baja lógica. Este campo permite desactivar registros sin eliminarlos físicamente de la base de datos, preservando la información histórica y evitando conflictos referenciales que podrían surgir si se utilizara una baja física.

En cuanto al diseño general, los atributos definidos en el UML fueron seleccionados para coincidir exactamente con los utilizados en el esquema SQL del trabajo anterior. En Empresa se representan

datos como razón social, CUIT, email y actividad principal, mientras que en DomicilioFiscal se incluyen calle, número, provincia, ciudad y código postal. Esta correspondencia garantiza coherencia entre la capa de dominio en Java, el script SQL de creación de tablas y las validaciones implementadas en la capa Service, permitiendo que todas las partes del sistema trabajen con el mismo modelo de información.

2.2. Justificación de la Relación 1→1 mediante FK única

Para implementar la relación uno a uno entre Empresa y DomicilioFiscal se optó por utilizar una clave foránea única en la tabla de domicilio fiscal. Con esta estructura, el campo *empresa_id* actúa simultáneamente como referencia y como restricción de unicidad, lo que garantiza que cada domicilio fiscal esté asociado a una sola empresa y que no existan dos domicilios vinculados al mismo registro. Esta solución es la forma más clara y estándar de modelar relaciones 1→1 dentro de bases de datos relacionales.

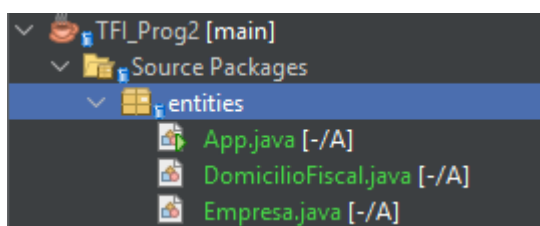
Además, esta decisión respeta la direccionalidad definida en el dominio: la empresa debe conocer su domicilio fiscal, pero el domicilio no necesita mantener una referencia inversa hacia la empresa. La implementación también permite utilizar correctamente reglas como *ON DELETE CASCADE*, lo que facilita las operaciones de baja lógica o física cuando corresponde. Finalmente, es un enfoque que se integra de manera natural con JDBC y los patrones DAO, ya que habilita un manejo ordenado de transacciones; por ejemplo, es posible insertar primero el domicilio fiscal y luego asignarlo a la empresa sin generar inconsistencias.

3. Modelado Java de Entidades

En esta etapa se desarrollaron las entidades Java correspondientes al modelo conceptual definido previamente: Empresa (A) y DomicilioFiscal (B). Ambas clases representan los objetos centrales del dominio “Empresa → DomicilioFiscal”, manteniendo una relación 1→1 unidireccional, donde una Empresa contiene a su Domicilio Fiscal.

El objetivo de esta etapa es construir clases limpias, coherentes con el diseño UML, aplicando buenas prácticas de encapsulamiento, legibilidad y mantenimiento.

3.1. Estructura de Paquetes



Las clases se encuentran dentro del directorio `src/entities`, específicamente en los archivos `Empresa.java` y `DomicilioFiscal.java`. Esta organización sigue la arquitectura por capas solicitada en la consigna, ya que permite mantener las entidades separadas del resto de los componentes del proyecto, como las capas DAO, Service, Config y Main.

3.2. Decisiones de Modelado

En el modelado se tomaron varias decisiones orientadas a mantener coherencia entre la capa de dominio en Java y el esquema relacional utilizado en el TFI de Bases de Datos I. Por ese motivo, cada clase incorpora exactamente los mismos atributos definidos en las tablas de MySQL, lo que asegura una correspondencia total entre ambos entornos. Además, ambas entidades incluyen el campo *eliminado*, que permite aplicar correctamente la baja lógica requerida para el trabajo práctico integrador.

También se definieron constructores según las necesidades del proyecto: uno vacío, indispensable para el funcionamiento de JDBC y diversos frameworks, y otro completo que facilita la creación de instancias con todos los datos necesarios, lo cual resulta útil tanto en pruebas como en inicialización del modelo. En cuanto a la relación entre las clases, se mantuvo la unidireccionalidad uno a uno establecida en la consigna. Por ello, solo la clase Empresa contiene una referencia a DomicilioFiscal, mientras que DomicilioFiscal no mantiene ningún vínculo hacia Empresa.

Finalmente, los métodos toString fueron diseñados para evitar recursión o impresiones circulares entre objetos. Cada uno devuelve únicamente la información relevante de la clase sin expandir representaciones completas de objetos asociados, lo que permite un funcionamiento más seguro y claro durante la depuración y el registro de datos.

3.3. Código de las clases

- *Empresa.java*
- *DomicilioFiscal.java*
- *App.java*

Salida esperada:

```
TFI_Prog2 (run) #4 × TFI_Prog2 (run) #5 ×
run:
DomicilioFiscal(id=1, calle='Av. Siempre Viva', numero=742, ciudad='Springfield', provincia='Buenos Aires', codigoPostal='1234', pais='Argentina')
Empresa(id=1, razonSocial='Mi Empresa SRL', cuit='30-99999999-9', actividadPrincipal='Software', email='info@empresa.com', domicilioFiscal=1)
BUILD SUCCESSFUL (total time: 0 seconds)
```

4. Persistencia y conexión a la base de datos (config/)

En esta etapa se implementó la capa de persistencia inicial del proyecto mediante la creación de la clase responsable de administrar la conexión con la base de datos MySQL utilizada en el Trabajo Final Integrador.

El objetivo principal fue configurar correctamente la comunicación entre la aplicación Java y la base tfi_bdi, previamente desarrollada en la materia Bases de Datos I. Para garantizar flexibilidad y buenas

prácticas, se incorporó un archivo externo de propiedades (`db.properties`) donde se parametrizan la URL de conexión, el usuario y la contraseña.

Esto permite modificar credenciales sin tener que editar el código Java.

4.1. Archivo de configuración: `db.properties`

Este archivo se ubicó dentro de la carpeta `config/` del proyecto NetBeans y contiene los parámetros utilizados por la clase de conexión.

```
5 db.url=jdbc:mysql://localhost:3306/tfi_bdi?useSSL=false&allowPublicKeyRetrieval=true
6 db.user=root
7 db.password=admin123
8 -
```

El uso de este archivo contribuye a una arquitectura más limpia, desacoplada y fácil de mantener.

4.2. Clase de conexión: `ConexionBD.java`

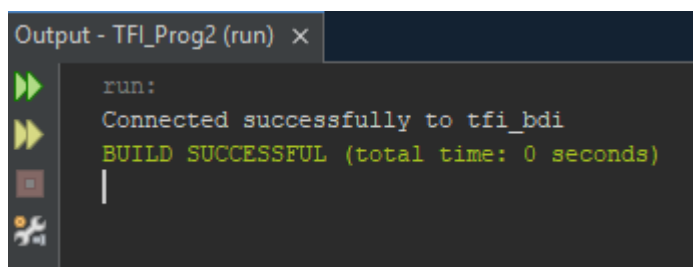
La clase `ConexionBD.java` se desarrolló con el objetivo de centralizar todo el proceso de acceso a la base de datos. Su función principal es leer los valores configurados en el archivo `db.properties`, registrar el driver JDBC de MySQL y establecer una conexión válida con el servidor. Además, se encarga de manejar las excepciones que puedan producirse durante este proceso y generar mensajes claros para facilitar la identificación de errores. De esta manera, garantiza una comunicación ordenada y confiable entre la aplicación y la base de datos.

4.3. Prueba de funcionamiento

Para validar la disponibilidad del servidor y la correcta lectura del archivo de propiedades, se creó una clase de prueba con un método `main()`.

4.4. Resultado obtenido

La ejecución se realizó desde NetBeans utilizando el servidor MySQL configurado previamente. Se obtuvo el siguiente mensaje en la consola:



```
Output - TFI_Prog2 (run) x
run:
Connected successfully to tfi_bdi
BUILD SUCCESSFUL (total time: 0 seconds)
```

Este resultado confirma que el driver JDBC está instalado correctamente, que el servidor MySQL se encuentra en funcionamiento en `localhost` y que la base de datos `tfi_bdi` existe y puede ser accedida sin inconvenientes. También demuestra que la configuración almacenada en la carpeta `config` funciona adecuadamente y es interpretada de forma correcta por la aplicación.

En términos técnicos, esta etapa permitió dejar totalmente operativa la capa de conexión con la base de datos, un paso indispensable para avanzar hacia las fases siguientes del proyecto, como la implementación de los DAOs, la capa de servicios y la persistencia real de los datos. Además, se estableció una arquitectura ordenada en la que los parámetros de conexión se encuentran aislados en el archivo db.properties, la lógica de conexión está concentrada en la clase ConexionBD, y existe una clase de prueba destinada a validar el proceso. Con esta estructura, la aplicación queda preparada para incorporar operaciones CRUD y manejar transacciones de manera adecuada en los próximos pasos del desarrollo.

5. DAO (Data Access)

En esta etapa, implementamos el acceso a los datos utilizando el patrón de diseño DAO (Data Access Object). Los DAOs que desarrollamos son responsables de interactuar con la base de datos para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre las entidades Empresa y DomicilioFiscal.

5.1 Descripción de los DAOs

La interfaz GenericDAO<T> se diseñó como una estructura genérica que reúne las operaciones CRUD fundamentales que serán utilizadas por todos los DAOs del proyecto. A través de ella se definen los métodos necesarios para insertar nuevos registros, consultar datos por ID, obtener todos los registros disponibles, actualizar información existente y aplicar eliminaciones lógicas mediante un campo indicador. Esta interfaz establece una base común que permite mantener coherencia y reutilización de código en todas las clases DAO que se implementen posteriormente.

A partir de esta estructura, se desarrolló EmpresaDAO, encargado específicamente de manejar la persistencia de las empresas en la base de datos. Este DAO incluye los métodos necesarios para registrar nuevas empresas, actualizar sus datos, aplicar eliminaciones lógicas en función del campo *eliminado* y consultar tanto empresas individuales como listados completos. Su implementación asegura que toda la interacción con la tabla correspondiente se realice de forma ordenada y consistente.

Del mismo modo, se creó DomicilioFiscalDAO, cuya responsabilidad es administrar la tabla de domicilios fiscales. Esta clase permite realizar las operaciones CRUD sobre los domicilios asociados a una empresa, garantizando la correcta relación mediante la clave foránea *empresa_id*. Con su implementación se logra mantener la integridad entre empresas y domicilios fiscales, asegurando que cada domicilio esté correctamente vinculado y que las operaciones respeten las reglas del modelo de datos definido.

5.2. Métodos CRUD implementados

Los DAOs desarrollados implementan las operaciones CRUD (Crear, Leer, Actualizar y Eliminar lógicamente) utilizando consultas SQL preparadas, lo que permite una interacción segura y eficiente con la base de datos.

Método crear:

Inserta un nuevo registro en la base de datos.

- En EmpresaDAO, el método crear() registra una nueva empresa y obtiene su ID generado automáticamente mediante getGeneratedKeys().
- En DomicilioFiscalDAO, el método crearConEmpresald() permite insertar un domicilio fiscal asociado a una empresa específica utilizando la clave foránea empresa_id.

Método leerPorId: Recupera un registro individual utilizando su identificador único, siempre que el campo eliminado sea false. Esto permite obtener, por ejemplo, una empresa o un domicilio fiscal activo en la base de datos.

Método leerTodos: Devuelve una lista con todos los registros activos de la tabla correspondiente, es decir, aquellos cuyo campo eliminado no ha sido marcado como true. Esto facilita obtener listados completos de empresas o domicilios fiscales.

Método actualizar: Modifica los datos de un registro existente. Cada DAO actualiza únicamente los campos correspondientes a su entidad, garantizando integridad y consistencia en las modificaciones realizadas.

Método eliminarLógico: Marca un registro como eliminado sin borrarlo físicamente. Esto se implementa cambiando el valor del campo eliminado a true, permitiendo mantener un historial sin pérdida de información.

5.3. Código de los DAOs

- *GenericDao.java*
- *EmpresaDao.java*
- *DomicilioFiscalDao.java*

5.4. Conclusión

La implementación de los DAOs permite realizar operaciones CRUD sobre las tablas empresa y domicilios_fiscales de forma eficiente, utilizando PreparedStatement para protegerse contra inyecciones SQL y mejorar el rendimiento. Las pruebas realizadas confirmaron que los métodos de inserción, lectura, actualización y eliminación lógica funcionan correctamente.

Gracias a la separación en DAOs, el acceso a datos queda desacoplado de la lógica de negocio, facilitando la escalabilidad y el mantenimiento del proyecto.

6. Service (Transacciones)

En esta etapa se implementa la capa de servicio encargada de administrar las transacciones y coordinar la interacción entre los DAOs. El servicio EmpresaService centraliza la lógica necesaria para crear una empresa junto con su domicilio fiscal asociado, garantizando que ambas operaciones se ejecuten dentro de una misma transacción. Para esto se desactiva el auto-commit de la conexión mediante `setAutoCommit(false)`, lo que permite controlar explícitamente cuándo confirmar los cambios con `commit()` o revertirlos con `rollback()` en caso de un error, como la detección de un CUIT duplicado.

6.1 Descripción del Servicio

El servicio EmpresaService contiene el método `crearEmpresaConDomicilioFiscal`, encargado de gestionar la creación conjunta de una empresa y su domicilio fiscal dentro de una única transacción. El proceso inicia con la validación de los datos de la empresa y la desactivación del auto-commit, lo que impide que los cambios en la base de datos se confirmen de manera automática.

Una vez iniciada la transacción, el servicio verifica que el CUIT no exista previamente en la base de datos. Si el CUIT es válido y no está duplicado, se procede a insertar la empresa y posteriormente su domicilio fiscal asociado, utilizando los métodos correspondientes de los DAOs. Si ambas operaciones se realizan exitosamente, se confirma la transacción mediante un commit, asegurando que los datos queden almacenados de manera definitiva.

Si ocurre algún error —ya sea por un CUIT duplicado o por una falla en el registro del domicilio fiscal— se ejecuta un rollback, lo que revierte completamente cualquier modificación realizada durante la transacción y evita que queden datos incompletos o inconsistentes. Finalmente, se restablece el auto-commit para que las operaciones posteriores vuelvan a comportarse de forma habitual.

6.2. Simulación de Rollback por CUIT Duplicado

Para demostrar el funcionamiento del mecanismo de rollback, se incluye una prueba en la clase `RollbackSimulado`. En este ejemplo se intenta registrar una empresa utilizando un CUIT que ya existe en la base de datos, como "30-99999999-9". El sistema detecta la duplicidad durante el proceso de inserción y lanza una excepción `SQLException`, lo que activa automáticamente el rollback dentro del servicio.

Antes de realizar el intento fallido, se crea un objeto `DomicilioFiscal` que se asocia a la empresa a registrar. Al producirse la excepción por el CUIT duplicado, la transacción completa es revertida, asegurando que no se inserte ni la empresa ni su domicilio. Este comportamiento confirma que el manejo de errores y la lógica transaccional funcionan adecuadamente y que el sistema garantiza la consistencia de los datos incluso en situaciones de falla.

6.3. Conclusión

La implementación del servicio y el manejo manual de transacciones permiten garantizar que la creación de una empresa y su domicilio fiscal sea un proceso atómico y coherente. Gracias al uso de `setAutoCommit(false)`, `commit()` y `rollback()`, se evita el almacenamiento parcial de datos y se protege la integridad de la información. En particular, la simulación del CUIT duplicado demuestra que ante cualquier excepción la transacción se revierte correctamente, evitando inconsistencias en la base de datos y asegurando un funcionamiento confiable del sistema.

7. AppMenu (Interfaz de Consola)

Esta etapa corresponde al desarrollo de la interfaz de consola que permite al usuario interactuar con la aplicación mediante un menú CRUD aplicado a la entidad principal *Empresa* y su relación uno a uno con *DomicilioFiscal*. Desde este menú se gestionan las operaciones básicas del sistema, mientras que internamente se utilizan los métodos proporcionados por la capa Service, encargada de coordinar la lógica de negocio y la comunicación con los DAOs.

7.1. Funcionalidades del AppMenu

```
run:
Connected successfully to tfi_bdi

*** MENÚ DE OPCIONES ***
1?? Crear Empresa
2?? Listar Empresas
3?? Modificar Empresa
4?? Eliminar (baja lógica) Empresa
5?? Buscar Empresa por CUIT
6?? Salir
Elige una opción:
```

La clase *AppMenu.java* implementa un menú interactivo que ofrece todas las funciones necesarias para administrar empresas y sus domicilios fiscales desde la terminal. A través de esta interfaz es posible crear nuevas empresas, ingresando tanto sus datos principales como los del domicilio fiscal correspondiente. Esta operación se realiza dentro de una transacción controlada por la capa de servicio, lo que garantiza que la inserción de ambos registros se lleve a cabo de manera coherente y sin inconsistencias.

El menú también permite listar todas las empresas activas, es decir, aquellas que no fueron marcadas como eliminadas lógicamente. Además, ofrece la opción de modificar información específica, como el correo electrónico, siempre validando previamente la existencia de la empresa seleccionada. Asimismo, se incluye la posibilidad de aplicar una baja lógica, marcando una empresa como eliminada sin borrar físicamente sus datos, lo que contribuye a preservar la integridad histórica del sistema.

Otra funcionalidad disponible es la búsqueda de una empresa a partir de su CUIT, permitiendo acceder de forma rápida y directa a los registros correspondientes y contemplando los casos en los que el dato no exista en la base. Todo el menú incorpora validaciones para evitar errores comunes, junto con un manejo adecuado de excepciones, como el ingreso de un ID inválido o fallas en la transacción. Además, se presentan mensajes claros y orientativos que facilitan la interacción del usuario de manera sencilla y segura. Ejemplos de salida:

```
Elige una opción: 1

*** Crear Empresa ***
Razón Social: Malibu SA
CUIT: 30-45123632-5
Actividad Principal: Ganaderia
Email: malibu@info.com

Ingrese los datos del Domicilio Fiscal:
Calle: Laprida
Número: 456
Ciudad: CABA
Provincia: BSAS
Código Postal: 1456
País: ARGENTINA
Empresa y domicilio fiscal creados correctamente.
```

```
Elige una opción: 2

*** Listar Empresas ***
Empresa{id=7, razonSocial='Malibu SA', cuit='30-45123632-5', actividadPrincipal='Ganaderia', email='malibu@info.com', domicilioFiscal=5}
```

```
Elige una opción: 3

*** Modificar Empresa ***
Ingrese el ID de la empresa a modificar: 7
Empresa encontrada: Empresa{id=7, razonSocial='Malibu SA', cuit='30-45123632-5', actividadPrincipal='Ganaderia', email='malibu@info.com', domicilioFiscal=5}
Nuevo Email (dejar vacío para no cambiar): malibuinformacion@gmail.com
Empresa actualizada correctamente.
```

```
Elige una opción: 4

*** Eliminar Empresa (Baja Lógica) ***
Ingrese el ID de la empresa a eliminar: 7
Empresa eliminada lógicamente.
```

```
Elige una opción: 5

*** Buscar Empresa por CUIT ***
Ingrese el CUIT de la empresa: 30-45123632-5
Empresa encontrada: Empresa{id=7, razonSocial='Malibu SA', cuit='30-45123632-5', actividadPrincipal='Ganaderia', email='malibuinformacion@gmail.com', domicilioFiscal=5}
```

7.2. Conclusión y posibles mejoras a futuro

La implementación de *AppMenu* representa la capa de interacción directa con el usuario y permite comprobar el funcionamiento integral del CRUD y de la relación uno a uno entre empresa y domicilio fiscal. Al centralizar las operaciones y delegar la lógica al servicio, se verifica el correcto acoplamiento entre todos los componentes del sistema: las entidades que modelan los datos, los DAOs encargados del acceso mediante JDBC, la capa de servicio que administra las transacciones y la interfaz de consola que facilita la ejecución de cada operación. Esta etapa evidencia que la aplicación funciona de manera consistente y que todas sus capas se encuentran correctamente coordinadas, asegurando un flujo de trabajo claro, ordenado y confiable.

En cuanto a posibles mejoras futuras, la aplicación podría ampliarse incorporando un sistema de validaciones más robusto para los datos ingresados por el usuario, una capa de presentación gráfica utilizando tecnologías como JavaFX o Swing, o incluso una API REST que permita interactuar con la lógica del sistema desde aplicaciones externas. Asimismo, sería posible extender el modelo de datos incorporando nuevas entidades relacionadas, agregar reportes o funcionalidades de auditoría y mejorar el manejo de excepciones para ofrecer mensajes aún más específicos y detallados. Estas

mejoras potenciarían la escalabilidad del sistema y permitirían una experiencia de uso más completa y moderna.

ANEXO

Roles de los integrantes del equipo

- **Camilo Capelli:** Responsable del diseño y armado completo de la estructura SQL, incluyendo el modelo de tablas y la relación 1→1 utilizada por la aplicación.
- **Ramón Armando Montero:** Encargado del desarrollo en Java, implementando las entidades, DAOs, capa Service, AppMenu y la integración total con la base de datos mediante JDBC.
- **Trabajo conjunto:** Ambos realizaron en conjunto el video explicativo y la elaboración del informe final en PDF.

Fuentes utilizadas

- **Materiales de la cátedra Programación II – UTN:** diapositivas, apuntes y ejemplos brindados por la profesora y el tutor.
- **Apuntes de Bases de Datos I – UTN:** normalización, restricciones, diseño relacional y conceptos de integridad referencial.
- **GitHub:** repositorio utilizado para control de versiones y trabajo colaborativo.
- **Herramientas de asistencia basadas en IA (ChatGPT):** usadas únicamente para mejorar redacción y organización del informe, manteniendo la implementación original del equipo.

Conclusión final del trabajo

El proyecto permitió integrar de forma práctica los contenidos de Programación II y Bases de Datos I, desarrollando una aplicación completa con relación 1→1, manejo transaccional, persistencia con JDBC y un menú CRUD funcional. Mediante el trabajo colaborativo se logró un sistema coherente, organizado y técnicamente correcto, aplicando buenas prácticas de diseño, programación y modelado.